CPSC 441 – Computer Networks – Fall 2026
Programming Assignment 1

Due Date: October 13, 2026, at 11:59 pm

***Late penalty: 20% deduction per day**

- This is an **individual** assignment. Codes **must** be written individually.
- Recommended programming languages are **Python** and **Java** (others may be used only after approval from the course instructor, but they are not recommended since TA support for them may be limited). Once selected, you **must** use it to code the entire assignment. Your assignment cannot be written in different languages.
- The source code and project report are to be uploaded to the D2L shell by the deadline.
- The use of ChatGPT or other generative AI tools to generate code is **Not allowed**. However, you may use ChatGPT or other generative AI tools to support your learning in appropriate ways, such as asking for clarification of programming concepts, explaining error messages, or suggesting strategies for debugging and testing your own code. Keep in mind that these tools can sometimes produce incorrect or misleading answers.
- You have a support system in this course, and you are always more than welcome to reach out to get all the help and support you need. You can always reach out to the TAs to schedule office hours or go to them during their designated tutorial slots for help and support.

Deliverables

---

1. A Report detailing (1-3 pages):
    a. Brief description of the code operation, steps to run the code, and assumptions/considerations (if there are any assumptions/considerations). Note: Be sure to mention anything that might not be immediately clear, so the TAs can fully understand and fairly grade your work.
    b. Difficulties you faced and how you handled them (if any)
    c. Possible improvements: If you had more time, what would you add or do differently?

2. Source code (Use comments to document/describe your code)

Description

Programming assignments are meant to help you better understand the networking concepts taught throughout the course.

As a start, in this assignment, you will be required to implement a simple client-server communication application. Detailed requirements are found below.

Requirements

1. Design a client-server communication application (chatting application between clients and server).
2. Clients and server are to communicate using **sockets ONLY** (TCP). You will launch **only one server** and as many clients as the server can serve.
3. Once created, clients are assigned a name in the form of ["Client" + incremental number starting with 1]. That is, first client name will be "Client01", second client is "Client02", and so on.
4. Once the connection is accepted, the client will communicate their name to the server.
5. The server will maintain a cache of accepted clients during current session along with date and time accepted, and date and time finished. Cache will be only in memory, no need to use files.
6. A server can handle a limited number of clients (you will hard code or configure this number, **let's assume 3 clients**).
7. Points 3 and 6 are easier to be implemented on the server side. But you can choose either side.
8. Once connected and its name sent to the server, a client can send any string using CLI (Command Line Interface). The server on the other side should echo the same string with the word "ACK" attached.
9. Once a client finishes sending messages, it can send an "exit" message to terminate the connection. On the server side, and upon receiving a connection closure request, the server closes the connection to free resources for other clients.
10. The server will have a repository of files. After accepting the client's connection, the client sends the "list" message to the server. The server should then reply with the list of files in its repo. The client can send the name of any file to the server, and the server should stream the file to the client. You should handle all cases, like sending a file name that does not exist.

If there is a detail not included in the requirements above, then it is totally up to you to decide on. However, all your code design considerations must be detailed in the "Brief Description" section of the report. In the "Possible Improvement" section, you can detail all the things you would want to do if you had more time.

Rubric

Rubric: (No partial credit under each item, it is either fully met or receives no points)

1.	Program can create a Server and client	2 points
2.	Each client created is assigned a name with correct number	1 point
3.	Server can handle multiple clients (Multi-threading)	2 points
4.	Server is able to force the number of connected clients to 3 clients	2 points
5.	Server and client can exchange messages as described	2 points
6.	A client can send "exit" and server cleanly disconnects the client for new clients	2 points
7.	Server maintains clients' connections details	1 point
8.	Server is able to list files in its repository	1 point
9.	Server streams files of choice to clients	2 points

Note:

- In exceptional cases, if you submit the required report and source code and demonstrate significant effort in the right direction, partial credit **may be** awarded. Please do not let the 'No partial credit' rule discourage you from doing your best, making an effort, and showing it.

Sample test cases

Make sure to test all items in the rubric.